

Selection Beats Sophistication: Discovering Interpretable Creativity Metrics with Adversarial Agent Evolution

Anonymous Submission

Abstract

Judging whether one piece of writing is more *creative* than another is easy for people and surprisingly hard to automate with anything transparent. Large language models can act as judges, but they are slow, expensive, and opaque. We ask whether a small team of language-model *agents* can instead **discover** short, fast, fully inspectable scoring functions — ordinary Python that anyone can read — that rank a fresher rewrite of a text above a flatter one. Our system pits two agent roles against each other: an *engineer* that writes new scoring functions after studying where the current best one fails, and an *adversary* that writes new test cases designed to fool the current best. A deterministic referee filters every proposal through four automatic gates: it must behave correctly, must not memorize the test set, must behave differently from every existing scorer, and must genuinely separate better rewrites from worse ones. Running this loop, the system rediscovered — without being told — a classical principle from creativity research: a good creativity measure must combine a novelty signal with a sanity check, multiplicatively. We then ran the same loop with a much smaller model generating three times as many proposals. The smaller model's individual ideas were worse, yet the loop converged to an equally strong final scorer. Acceptance rates were identical under both models, and both runs converged on the same design shape, suggesting that the selection pressure — not the sophistication of the proposing model — is the primary driver of discovery. We report the full trajectories, the failure modes our honesty checks caught (including a champion that secretly overfit the test distribution and a test-set-growth rule that backfired), and the ceiling we hit: word-statistics can measure freshness, but judging whether a fresh image actually *fits* still requires meaning-aware tools.

1 Introduction

Suppose an editor rewrites the sentence “*The storm arrived late at night*” as “*Night cracked open into a storm.*” Most readers agree the second is more creative. Now suppose we want a program to make that call — millions of times, cheaply, and in a way we can audit. This is the problem of **scoring subjective text quality**, and creativity is among its hardest instances.

The fashionable answer is to ask a large language model (LLM) to be the judge. That works, but the judge is a black box: we cannot inspect *why* it preferred one text, we cannot run it offline in microseconds, and its preferences drift with the model version. The classical answer is to hand-craft a metric — count rare words, count clichés — but hand-crafted metrics encode their author's blind spots and, as we show, collapse the moment a test case decouples the proxy from the quality it stands for.

This paper explores a third path: let LLM agents **write the metric as code**, and let an evolutionary process — selection on measured performance, plus an adversary that continually manufactures harder test cases — decide what survives. The output is not a neural network but a short, deterministic Python function anyone can read. Two questions drive the work:

Q1. Can such a loop *discover* a genuinely novel, robust scoring principle, rather than just re-tuning known ones?

Q2. When it succeeds, what deserves the credit — the intelligence of the model writing the proposals, or the evolutionary scaffolding around it? We test this directly by re-running the identical loop with a far smaller model given three times the proposal budget.

Our answers, in brief: yes (the loop independently reinvented the field's standard two-factor definition of creativity as an executable program), and — more surprisingly — **the scaffolding carries most of the weight**. The smaller model's proposals were individually weaker and more repetitive, but the acceptance rate after filtering was identical, and its final champion matched or beat the larger model's on every honest measure we track.

2 The Task, in Plain Terms

What a scorer is. A scorer is a Python function `score(text, original)` returning a number between 0 and 1. It sees a rewrite and the original it came from, and should give higher numbers to rewrites that are more creative. It must be deterministic (same input, same output), must not call any network or model, and may use only ordinary computation plus one public resource: a table of how frequent each English word is in large text corpora.

What a test case is. Each test case is a triple: an original two-sentence passage, a rewrite that a careful human would call *more* creative, and a rewrite they would call *less* creative — all describing the same facts. A scorer handles the case correctly if it gives the more creative version the higher score.

How we measure success. Accuracy (fraction of cases ranked correctly) hides too much: a scorer that squeaks past by 0.001 on every case looks identical to one that separates confidently. We therefore score both versions of every case individually and use the **separation**: the average score of all the better rewrites minus the average score of all the worse ones. A separation of 0 means the scorer cannot tell them apart at all; 1 is a perfect, maximally confident split. We report accuracy and the two averages alongside it, so over-

and under-confidence are both visible.

Why normalization matters. Every scorer must map to the same 0–1 scale, but calibrating that scale on the test cases themselves would be cheating. Scorers instead calibrate against fixed reference statistics computed once from the public word-frequency table (for example, how surprising the average English word is), so the scale is anchored to the language, not to our data.

3 Method: an Adversarial Evolutionary Loop

The system has one deterministic referee (ordinary code, no model) and two agent roles, each played by an LLM with its own private context.

The engineer proposes new scorers. Each generation it receives: a map of creativity concepts distilled from the research literature (which concepts existing scorers already measure and which are unclaimed), a statistics table for every current scorer, a similarity matrix showing which scorers behave alike, the champion's code, and the specific training cases the champion gets wrong. It must write complete, runnable modules — reasoning first, code second.

The adversary proposes new test cases. It sees only the champion's code and a list of attack angles already used, and must invent a *new* way to decouple the champion's signal from real creativity — for example, if the champion rewards rare words, write flat rewrites full of stiff formal vocabulary (high signal, no creativity) and brilliant rewrites made of everyday words (low signal, high creativity). Candidate cases are kept only if the champion actually struggles on them; most join the training set and a fixed share is diverted, sight unseen, into a held-out set.

Four acceptance gates. Every proposed scorer passes through automatic checks before it may join the population:

1. **Validity** — it runs without error, stays within 0–1, is deterministic, and is not near-constant.
2. **No memorization** — the referee parses the code and inspects every hard-coded word list; if it contains rare words copied from the visible test cases, or any large content word list at all, the scorer is rejected. (This gate exists because, in a pilot, an agent shown failing cases quietly copied eleven of their distinctive verbs into its feature list, inflating its measured skill.)
3. **Behavioral novelty** — we record, for every scorer, its per-case score difference between the better and worse rewrite. If a new proposal's pattern of differences correlates above 0.6 with any existing scorer's, it is the same signal in new clothes and earns no novelty credit, no matter how different its stated mechanism sounds.
4. **Usefulness** — it must beat the population's median separation, or be genuinely novel with positive separation.

Three tiers of test data keep everyone honest. The *training* cases are visible to agents and drive selection. The *held-out* cases are never shown to any agent and exist only

so we can check whether a champion's skill is real. The *stationary core* is a small, never-growing subset of the original cases, giving one number that stays comparable across all generations even as the test set grows harder.

Schedule. The population holds at most ten scorers: the six fittest, up to two protected slots for the most behaviorally unusual, and the newest arrivals. Each generation adds a batch of engineer proposals and adversary cases; the loop stops when two consecutive generations bring neither a better champion nor an accepted novel scorer. The initial population is four baseline scorers implementing standard ideas from the creativity literature: word rarity, unusual word combinations, an “is this overdone?” sanity check, and their product.

4 Experiments

Arm A (larger model, low volume). A mid-size commercial LLM played both roles for three generations: 3 scorer proposals and 8 adversarial candidates per generation (6 kept).

Arm B (smaller model, high volume). A much smaller, cheaper LLM played both roles under an identical harness, seeds, and gates, but at three times the mutation volume — 9 proposals per generation from three parallel engineers with different briefs, and 16 adversarial candidates (10 kept) from two parallel adversaries — and ran for seven agent generations. Everything else was held fixed.

Both arms start from the same 42 test cases (20 built from real internet text plus 22 from two earlier adversarial rounds), split deterministically into 29 training and 13 held-out cases.

	gen	train	champion (best scorer)	sep.	held-out	core
A	0	29	unusual-combination baseline	0.072	0.059	0.212
A	1	33	added-detail structure (agent)	0.032	0.018	−0.003
A	2	37	kept-skeleton + fresh insert (agent)	0.129	0.161	0.202
A	3	41	(same champion holds)	0.074	0.121	0.202
B	0	29	unusual-combination baseline	0.072	0.059	0.212
B	1	36	insert-vs-swap structure (agent)	0.074	0.098	0.135
B	2	43	short-word concreteness (agent)	0.201	0.190	−0.009
B	3	50	(reverts to gen-1 champion)	0.055	0.068	0.135
B	4	57	variety × rarity gate (agent)	0.106	0.158	0.359
B	5	63	(same champion holds)	0.139	0.187	0.359
B	6	70	(same champion holds)	0.123	0.169	0.359
B	7	77	(same champion holds)	0.154	0.198	0.359

Table 1: Champion trajectory in both arms. “sep.” is separation on the (growing) training set; “held-out” is separation on cases no agent ever saw; “core” is the never-growing stationary subset. Agent-written champions displaced the baselines in both arms.

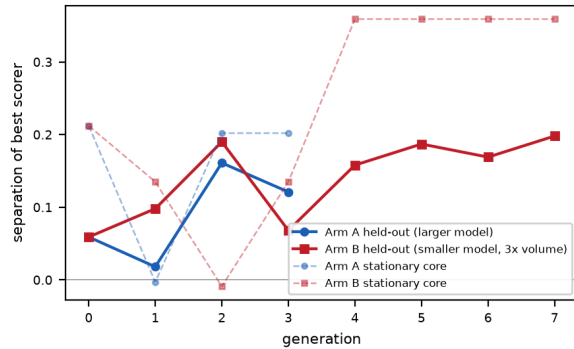


Figure 1: Held-out separation (solid) and stationary-core separation (dashed) of each generation's champion. Arm B is chaotic early — including a generation-2 champion whose core score went *negative*, i.e., it had overfit the adversarial distribution — then converges to the strongest scorer of either arm.

5 Results

5.1 A Classical Principle, Rediscovered as Code

Creativity research has long held that creativity is not novelty alone: an idea must be both *new* and *fitting* (Runco and Jaeger 2012), and preference for novelty peaks at moderate levels rather than growing without bound (Berlyne 1971). Nobody told the agents this. Yet in both arms, every surviving champion has the same architecture: a novelty signal **multiplied by a sanity gate**. Arm A's discovery scores “how novel is the material you inserted, *given* that you kept the skeleton of the original intact”; Arm B's scores “how varied is the rewrite's structure, *given* that its content earns the variety.” Single-signal scorers — rarity alone, divergence alone, concreteness alone — were each destroyed by an adversarial round within one generation of taking the lead. The two-factor multiplicative shape was not seeded and was not suggested in any prompt; it emerged twice, independently, because it is the only shape the adversary could not decouple.

5.2 Model Quality Sets the Pace, Not the Destination

	Arm A (larger)	Arm B (smaller, 3× vol.)
agent generations	3	7
scorer proposals	9	45+
accepted	3 (33%)	~15 (33%)
copycats caught by gate 3	3	12+
champion turnovers	2	4 (one regression)
final champion held-out sep.	0.121	0.198
final champion core sep.	0.202	0.359
strongest adversary round	−0.68 avg. margin worst possible (−1.0) × 10	

Table 2: Head-to-head. The held-out sets diverge across arms after generation 0 (different adversaries grew them), so the stationary core — the same six cases in both arms — is the honest cross-arm comparison.

Three observations answer Q2.

(a) The gates, not the proposer, set the quality bar.

The acceptance rate was 33% under both models. The smaller model produced four times as many “copycats” —

proposals whose stated mechanism sounded new but whose behavior correlated up to 0.94 with an existing scorer — and the behavioral-novelty gate caught every one. Claimed mechanisms do not survive contact with measured behavior.

(b) Attacking is easier than building. The smaller model's adversaries matched and then exceeded the larger model's: one of its rounds achieved the worst possible margin on all ten kept cases, the hardest single hit in either arm. Finding a scorer's blind spot appears to demand less capability than designing a scorer.

(c) Volume plus generations compensate for weaker proposals. The smaller model's path was turbulent — a champion overturned every generation for three rounds, one overfit champion exposed by the stationary core, one outright regression — but at three times the proposal volume and with four extra generations it reached a final champion that beats the larger model's on the only strictly comparable number (core separation 0.359 vs. 0.202) and survived four consecutive targeted attack rounds.

5.3 What the Honesty Machinery Caught

The run is as interesting for what the referee *rejected* as for what it kept.

A memorizing scorer. In the pilot that motivated gate 2, an agent shown the champion's failures copied their distinctive vocabulary into its own word lists; its measured skill was partly a lookup table of the test set. The automated code-inspection gate now rejects this pattern outright.

A distribution-overfit champion. Arm B's generation-2 leader posted the best training separation seen to that point (0.201) while scoring *below zero* on the stationary core: it separated the adversary-authored cases while failing ordinary prose. Without a never-growing tier, this would have read as a breakthrough.

A test-growth rule that backfired. Our rule kept the ten candidate cases the champion handled *worst*. Once the champion became strong enough that every attack failed, “worst” cases were still ranked correctly — so the test set began absorbing champion-friendly cases and the champion's measured skill inflated. An adversarial test set only stays hard while the adversary stays competitive; the keep-rule must require the champion to actually fail, or discard the batch.

5.4 The Ceiling

The loop eventually converged by *adversary exhaustion*: in Arm B's final generation, neither of two fresh attack angles produced a single case the champion mis-ranked, and the engineers' remaining textbook ideas (a standard lexical-diversity algorithm, a statistical de-confounding trick, two-word-sequence surprise) each failed the gates on the merits. The concepts that stayed unmeasured — whether an inserted image actually *fits* its context, true meaning-level distance — are exactly the ones that word-frequency statistics cannot express. One adversary

demonstrated this pointedly, submitting pairs whose two rewrites are statistically near-identical but semantically opposed (an apt image versus a subtly impossible one). No scorer in the population can tell such pairs apart, and on the evidence here, none built from frequency statistics ever will. That boundary — freshness is measurable, fit is not — is, we think, the clearest empirical statement of where interpretable text metrics end and meaning-aware models must begin.

6 Related Work

Our loop belongs to the family of LLM-guided evolutionary search over programs, most prominently FunSearch (Romera-Paredes et al. 2024), which pairs an LLM proposer with a programmatic evaluator; we add an adversarial data-growth role and honesty gates targeted at the failure modes of subjective evaluation. Adversarially-collected benchmarks that grow with the model under test were popularized by Dynabench (Kiela et al. 2021). Program-space search by selection descends from genetic programming (Koza 1992). On the creativity-measurement side, we build on the two-factor definition (Runco and Jaeger 2012), moderate-novelty preference (Berlyne 1971), computational novelty measures based on semantic distance (Beaty and Johnson 2021; Olson et al. 2021; Johnson et al. 2022), and lexical-diversity measurement (McCarthy and Jarvis 2010). Our contribution is not a new creativity theory but evidence that a gated adversarial loop can re-derive the theory’s structure as executable code, and an ablation isolating how much the proposing model matters.

7 Limitations

The “more/less creative” labels were authored by LLMs, not humans; human ratings are the essential next validation. Each arm was run once — trajectory details (which champion falls when) are surely seed-dependent, though the convergent design shape appeared in both arms. The stationary core is only six cases. Separations are modest in absolute terms (0.1–0.4 on a 0–1 scale) because the test set is deliberately adversarial. Both arms received the same orchestrated lesson-passing between generations (each generation’s context summarizes what killed the last champion); we view this memory as part of the evolutionary machinery, but it means our ablation isolates the proposing model, not the orchestration. Finally, pre-trained semantic resources were unavailable in our sandboxed environment, which made the frequency-statistics ceiling unavoidable rather than chosen.

8 Conclusion

A small agent ensemble — one role writing scoring code, one role writing counter-examples, and a deterministic referee enforcing validity, non-memorization, behavioral novelty, and usefulness — twice rediscovered the two-factor structure of creativity as short, readable Python, and produced scorers that survive attacks their own authors

designed. Swapping the proposing model for one far smaller changed the journey but not the destination: identical acceptance rates, equal-or-better final champions, the same convergent design. The practical recipe we take away is that when searching for evaluation metrics with LLMs, **invest in the referee and the adversary before investing in a bigger proposer** — and always keep one test set the process is never allowed to touch.

References

- Beaty, R. E., and Johnson, D. R. 2021. Automating creativity assessment with SemDis. *Behavior Research Methods* 53:757–780.
- Berlyne, D. E. 1971. *Aesthetics and Psychobiology*. Appleton-Century-Crofts.
- Johnson, D. R.; Kaufman, J. C.; et al. 2022. Divergent semantic integration (DSI): Extracting creativity from narratives. *Behavior Research Methods* 55:3726–3759.
- Kiela, D.; Bartolo, M.; Nie, Y.; et al. 2021. Dynabench: Rethinking benchmarking in NLP. In *Proc. NAACL*, 4110–4124.
- Koza, J. R. 1992. *Genetic Programming: On the Programming of Computers by Means of Natural Selection*. MIT Press.
- McCarthy, P. M., and Jarvis, S. 2010. MTL-D, vocd-D, and HD-D: A validation study of sophisticated approaches to lexical diversity assessment. *Behavior Research Methods* 42:381–392.
- Olson, J. A.; Nahas, J.; Chmoulevitch, D.; Cropper, S. J.; and Webb, M. E. 2021. Naming unrelated words predicts creativity. *PNAS* 118(25).
- Romera-Paredes, B.; Barekatin, M.; Novikov, A.; et al. 2024. Mathematical discoveries from program search with large language models. *Nature* 625:468–475.
- Runco, M. A., and Jaeger, G. J. 2012. The standard definition of creativity. *Creativity Research Journal* 24(1):92–96.